Лабораторная работа №2

Выполнили студенты группы 932420:

Данила Аксёнов

Никита Теребинов

Николай Осипенко

1) Диаграмма классов

На основе МПО “СКУД”, сформированной в лабораторной работе №1, наша команда построила диаграмму классов, изображённую на рисунке 1.

В отличие от МПО, на диаграмме классов отсутствует класс Администратор (Administrator), т.к. было осознано, что хоть семантически (т.е. в реальном мире) он в любом случае будет существовать, но в создаваемом нами приложении он будет представлен всё-таки как Актёр (Actor), который управляет контентом системы (т.е. Сотрудниками, их Ключами и Правилами доступа, Зонами доступа и др.) посредством взаимодействием с конечным набором форм с GUI (графическим интерфейсом), как это было продемонстрировано на диаграмме анализа из лабораторной работы №1, а не как класс.

Как и требуется в ТЗ, на диаграмме классов присутствует абстрактный класс, которым является Ключ доступа (Key), неабстрактными потомками которого являются классы Card и Fingerprint, семантически являющиеся физической пластиковой магнитной картой доступа и отпечатком пальца соответственно. Ключ-карта (Card), в отличие от родительского класса Ключа доступа (Key), однозначно идентифицируется её уникальным номером (ID), а Отпечаток пальца (Fingerprint) — хэшем его рисунка (Hash).

Ещё одним существенным нововведением является приватный атрибут location у класса Сотрудника (Staff), который отображает, находится ли данный Сотрудник в одной из Зон доступа, и если да, то в какой. Данный

механизм позволяет обнаруживать ситуации, когда Сотрудник сначала зашёл в какую-то Зону доступа, а затем покинул её незаконным путём (или наоборот: зашёл незаконно, а выйти пытается с помощью одного из своих Ключей доступа) — тогда данному Сотруднику отказывается вообще в каком-либо доступе, и для его восстановления ему необходимо, например, написать объяснительную. Отследить двойное нарушение в виде несанкционированного входа в некоторую Зону доступа и такого же выхода из неё, конечно, не представляется возможным для конструируемой системы — эта обязанность возлагается на плечи охранного персонала и систем видеонаблюдения/-фиксации. Преимуществом и гибкостью предложенного механизма является то, что атрибут location привязан именно к Сотруднику, а не к одному из его Ключей доступа, что позволяет ему (Сотруднику) законно входить в Зону доступа и выходить из неё с использованием различных Ключей доступа. Функционал изменения атрибута location находится в методе того же класса SwitchLocation(controllerArea:Area), который вызывается только в конструкторе класса События прохода (Event) и только в случае положительного результата соответствующего запроса доступа.

Также на диаграмме классов были отображены встроенные в C# классы DateTime и TimeOnly, которые активно используются в логике предметной области ввиду их удобства. Соответствующие связи также были добавлены в диаграмму.

В соответствие с общепринятыми рекомендациями к стилизации кода в C#, названия всех публичных атрибутов и операций указаны в CamelCase, а имена всех приватных членов класса — в том же написании, но начинаются с маленькой буквы (camelCase).

Таблица 1 представляет из себя характеристику построенной диаграммы классов.

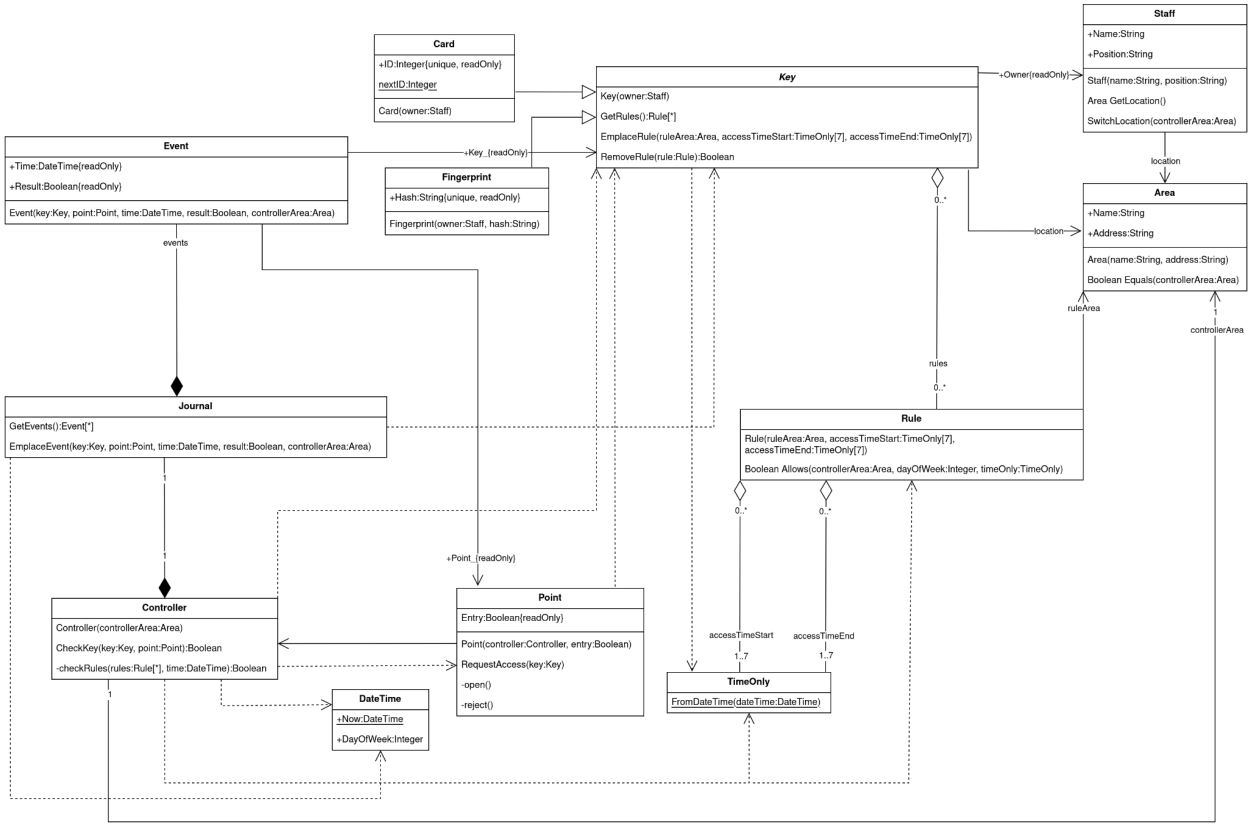


Рисунок 1 — диаграмма классов предметной области “СКУД”

Таблица 1 — характеристика диаграммы классов предметной области “СКУД”

Вопрос	Каким набором классов с каким содержимым (атрибуты, операции) и какими связями программно реализована предметная область “СКУД”?
Читатель	Разработчик, реализующий прямое проектирование системы.
Что сделает читатель?	Использует диаграмму как основу для реализации классов, их атрибутов, операций (но не их реализаций!) и связей
Что показано на схеме?	1. Абстрактный класс Key является общим предком для неабстрактных классов Card и Fingerprint, при этом вся логика работы с правилами доступа инкапсулирована именно в родительском классе Key (private, а не protected), а на потомков возложена лишь обязанность произвольным образом идентифицировать самих себя; 2. 2 последовательные композиции от класса Controller к классу Event через класс Journal для уточнения, что Журнал событий не может существовать без привязки к какому-то Контроллеру (и, на самом деле, наоборот ввиду кратности связи — 1 к 1) и что Событие прохода не может существовать вне Журнала событий (обратное

	утверждение, однако, неверно); 3. Кратность каждой из двух агрегаций от класса Rule к классу TimeOnly (именно в таком направлении) равна 1..7, чтобы показать, что Правило доступа не может не давать доступ в соответствующую Зону доступа ни в один из дней недели, т.к. тогда оно достаточно бессмысленно, при этом сверху эта величина ограничена числом 7, соответствующим количеству дней в неделе; 4. Многим атрибутам был присвоен статус {readOnly}, т.к. в их (атрибутов) контекстах это необходимо и соответствует реальному миру. Данный статус, однако, не мешает вызывать у этих атрибутов их собственные методы, которые могут изменить внутреннее состояние соответствующих объектов (например, {readOnly} атрибут Owner:Staff у класса Key и публичный метод SwitchLocation(controllerArea:Area) у класса Staff — подробнее см. ниже в диаграмме последовательностей сценария 3).
Что не показано?	1. Реализации операций (методов) классов — это обязанность диаграмм последовательностей; 2. Панель управления системой (“админка”), внешние сервисы, база данных и способы хранения и загрузки данных; 3. Обработка ошибок и исключительных ситуаций (отсутствие контроллера, ошибки журналирования, некорректные данные и др.); 4. Механизмы синхронизации и конкурентного доступа; 5. Нефункциональные требования (производительность, безопасность хранения биометрических данных, масштабируемость и отказоустойчивость).
Допущения	1. Не совсем понятно, как программно отслеживать коллизии хэшей Отпечатков пальцев и на кого должна быть возложена эта обязанность (хотя с идентификационным номерами Карт доступа всё решается просто с помощью статического nextID). Возможно, за это ответственен качественный алгоритм хэширования, в котором вероятность коллизий минимальна, или же какой-нибудь управляющий класс :FingerprintManager, который на диаграмме классов отсутствует (см. графу “*Что не показано?*” текущей таблицы); 2. В языке программирования C# нет инструментов для того, чтобы реализовать ограничения, накладываемые связями типа “композиция” — запрет на существование объектов одного класса вне/независимо от объектов другого класса (см. п. 2 в графе “*Что показано на схеме?*” текущей таблицы).

2) Диаграммы последовательностей

Для диаграмм последовательностей нашей командой были выбраны 3 наиболее важных для предметной области “СКУД” сценария:

Сценарий 1: запрос доступа от произвольного Ключа доступа (Key) к Точке доступа (Point). Соответствующие диаграмма последовательностей и её характеристика представлены на рисунке 1 (там же, в левом верхнем углу диаграммы, приведены название класса, его метода и его параметров) и в таблице 1 соответственно. Данная диаграмма является непосредственным следствием диаграммы коммуникаций из лабораторной работы №1.

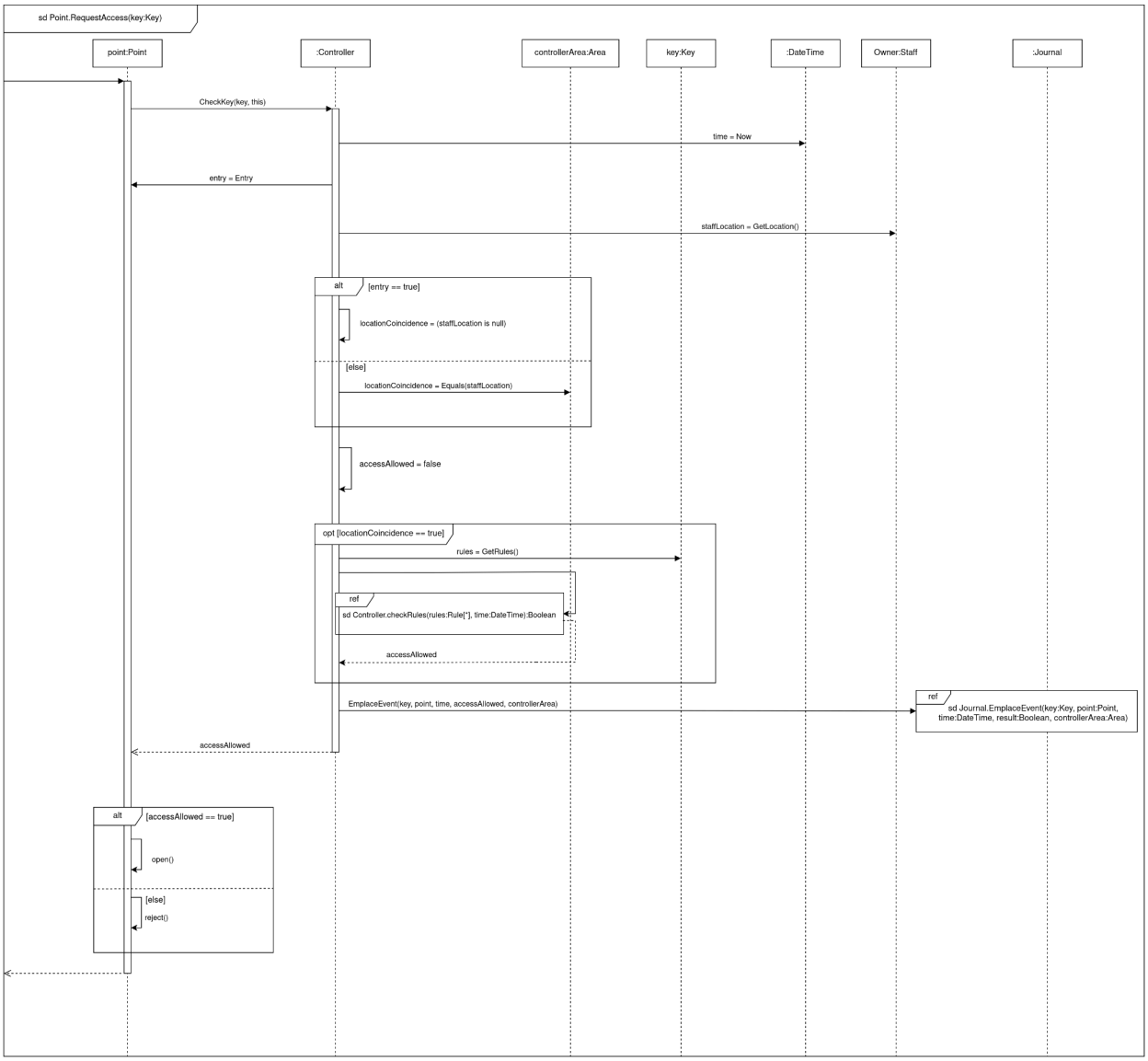


Рисунок 2 — диаграмма последовательностей сценария 1 (запроса доступа)

Таблица 2 — характеристика диаграммы последовательностей сценария 1

Вопрос	В соответствие с какой логикой система обрабатывает запрос доступа на проход через Точку доступа?
Читатель	1. Разработчик, реализующий прямое проектирование системы; 2. Сотрудник безопасности.
Что сделает читатель?	1. Программно реализует методы Point.RequestAccess(key:Key) и Controller.CheckKey(key:Key, point:Point):Boolean; 2. Поймёт логику работы системы и, возможно, выявит уязвимости.
Что показано на схеме?	1. Отчётливо видна проверка текущего местоположения Сотрудника и сопоставление его с тем, через Точку доступа какой Зоны доступа он пытается пройти и пытается ли он войти или же, наоборот, выйти; 2. По схеме становится ясно, что вся логика проверки Правил доступа делегирована Контроллеру, в то время как сама Точка доступа является лишь посредником между ним и Сотрудником с его Ключом доступа, который (посредник) выполняет его (Контроллера) поручения: дать Сотруднику пройти (open()) или же нет (reject()).
Что не показано?	На диаграмме последовательностей были упущены реализации: 1. методов Staff.GetLocation():Area, Area.Equals(area:Area):Boolean, Key.GetRules():Rule[*] ввиду их несложности; 2. методов Point.open() и Point.reject() ввиду того, что их логика является уже аппаратной, а не программной; 3. методов Controller.checkRules(rules:Rule[*]):Boolean и Journal.EmplaceEvent(key:Key, point:Point, time:DateTime, result:Boolean, controllerArea:Area) ввиду их относительной объёмности. Логика данных методов, как и указано в соответствующих ссылках, вынесены в отдельные диаграммы последовательностей (см. 2 последующий сценария).
Допущения	Не выявлены.

Сценарий 2: проверка списка Правил доступа (Rule) на соответствие текущему времени и Зоне доступа (Area), в которую запрашивается доступ из сценария 1. Соответствующая диаграмма последовательностей и её характеристика представлены на рисунке 2 и в таблице 2 соответственно. Данная диаграмма является раскрытием первой ссылки (*ref*) из диаграммы последовательностей сценария 1.

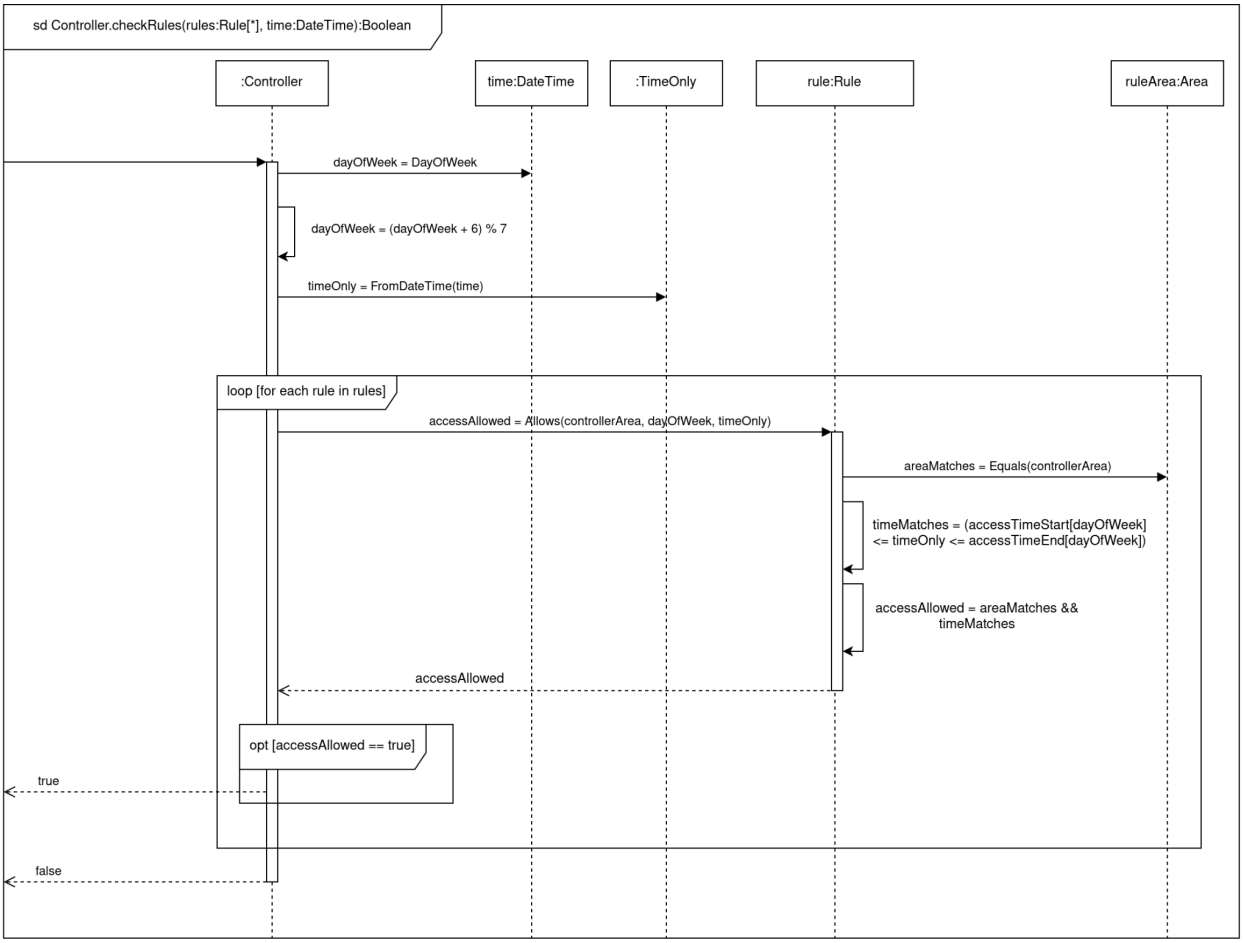


Рисунок 3 — диаграмма последовательностей сценария 2 (проверки списка правил)

Таблица 3 — характеристика диаграммы последовательностей сценария 2

Вопрос	Каким образом Контроллер проверяет существование хотя бы одного Правила доступа, разрешающего данному Сотруднику проход через его (Контроллера) Точки доступа в текущее время текущего дня недели?
Читатель	1. Разработчик, реализующий прямое проектирование системы; 2. Сотрудник безопасности.
Что сделает читатель?	1. Программно реализует методы Controller.checkRules(rules:Rule[*]):Boolean и Rule.Allows(controllerArea:Area, dayOfWeek:Integer, timeOnly:TimeOnly):Boolean; 2. Поймёт как Контроллер взаимодействует с текущими временем и днём недели, как он перебирает список Правил доступа, чем руководствуется отдельное Правило доступа при проверке себя на соответствие заданным времени и дню недели и, возможно, выявит

	уязвимости.
Что показано на схеме?	1. В языке программирования C# атрибут DateTime.DayOfWeek возвращает перечисление (enum), в котором воскресенье (sunday) кодируется числом 0 (свойственно западным странам), что вынуждает сдвинуть результат на единицу влево в кольце вычетов по модулю 7, чтобы перейти в привычный для нас календарь, в котором неделя начинается с понедельника; 2. На диаграмме указано, что Контроллер для проверки на соответствие данным условиям отдельного Правила доступа не нарушает инкапсуляцию класса Rule, а обращается к его публичному методу Rule.Allows(controllerArea:Area, dayOfWeek:Integer, timeOnly:TimeOnly):Boolean, в которой отдельно проверяется соответствие по Зоне доступа и по времени с днём недели, и положительный результат возвращается лишь в случае обеих успешных независимых проверок.
Что не показано?	1. Описание используемых методов у классов DateTime и TimeOnly, т.к. они в языке программирования C# они являются встроенными; 2. Реализация метода Area.Equals(area:Area):Boolean, как и в диаграмме последовательностей сценария 1, ввиду её/его (реализации/метода) относительной очевидности.
Допущения	Не выявлены.

Сценарий 3: добавление События прохода (Event) в Журнал событий (Journal) того Контроллера (Controller), в Зону доступа (Area) которого был запрошен доступ в сценарии 1. Соответствующая диаграмма последовательностей и её характеристика представлены на рисунке 4 и в таблице 4 соответственно. Данная диаграмма является раскрытием второй ссылки (*ref*) из диаграммы последовательностей сценария 1.

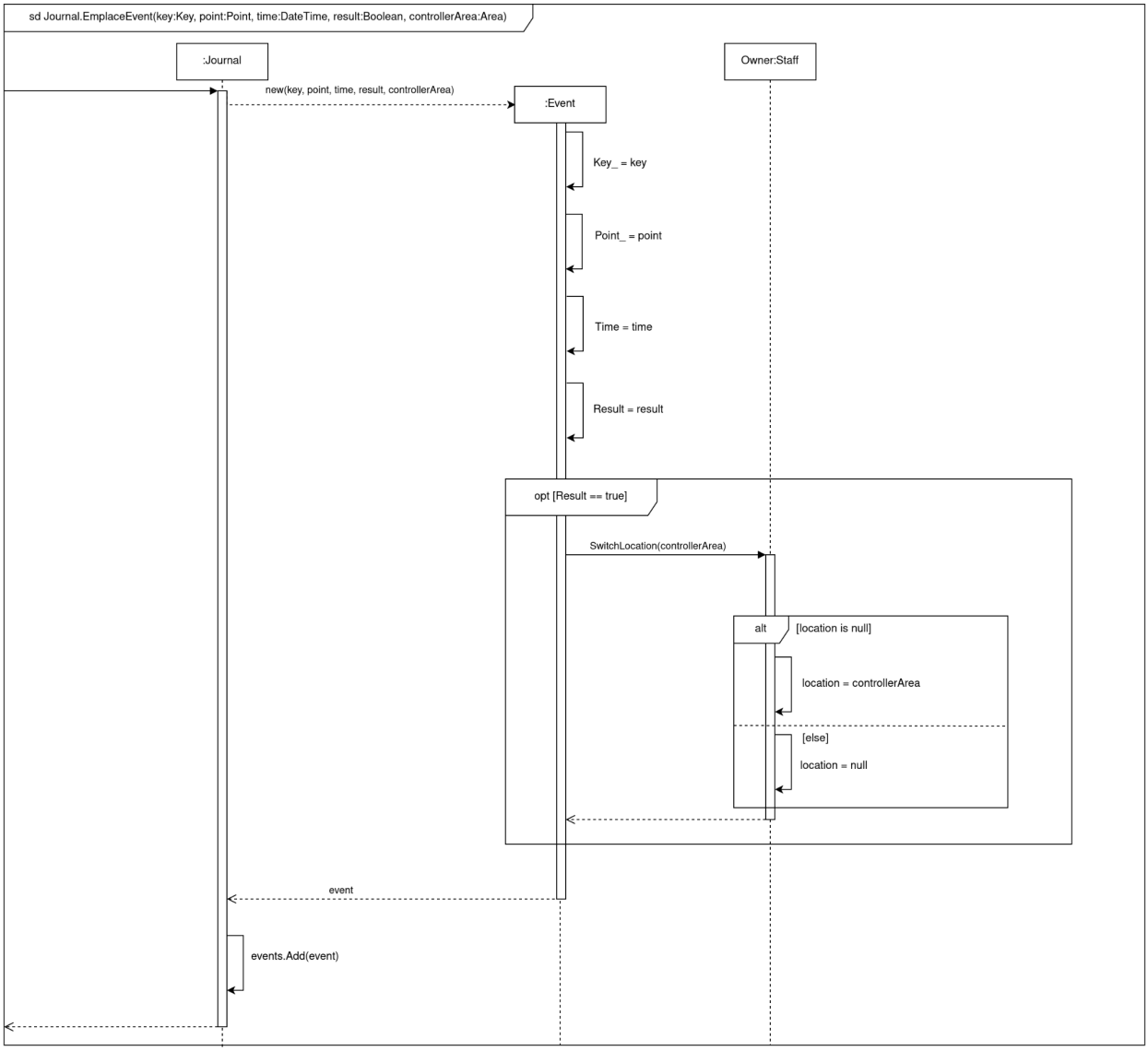


Рисунок 4 — диаграмма последовательностей сценария 3 (записи в Журнал событий)

Таблица 4 — характеристика диаграммы последовательностей сценария 3

Вопрос	Как Журнал событий регистрирует Событие прохода и при какие условиях это приводит к изменению местоположения Сотрудника?
Читатель	1. Разработчик, реализующий прямое проектирование системы; 2. Административное лицо.
Что сделает читатель?	1. Программно реализует методы Journal.EmplaceEvent(key:Key, point:Point, time:DateTime, result:Boolean, controllerArea:Area), Staff.SwitchLocation(controllerArea:Area) и конструктор Event(key:Key, point:Point, time:DateTime, result:Boolean,

	controllerArea:Area); 2. Поймёт, что могло привести к несоответствию местоположения Сотрудника в базе данных с его фактическим местоположением и более справедливо и/или объективно отнесётся к объяснительной данного Сотрудника и выберет необходимые меры в качестве наказания.
Что показано на схеме?	1. На схеме показано, что конструктор класса Event вызывается внутри метода класса Journal и, на самом деле, нигде более, что является решением, пусть и костыльным (т.к. ничто не запрещает вызвать этот же конструктор в другом месте программы), пункта 2 графы "*Допущения*" таблицы 1; 2. Видно, что обязанность поддерживать корректное местоположение Сотрудника возложена на конструктор класса Event, который делает это путём вызова метода Staff.SwitchLocation(controllerArea:Area), причём только в случае успешного прохода этого Сотрудника через выбранную им Точку доступа (Result == true).
Что не показано?	1. Механизмы поиска и фильтрации Событий прохода из Журнала событий, удовлетворяющих определённым критериям; 2. Правила обработки повторных запросов и определения дубликатов событий.
Допущения	Не выявлены.

3) Диаграмма объектов

На диаграмме объектов наша команда решила изобразить типичную взаимосвязь между классами Ключа доступа (Key) (если быть точнее, его наследника — Отпечатка пальца (Fingerprint)), Сотрудника (Staff), Правила доступа (Rule) и Зоны доступа (Area). В качестве примера используется один из членов нашей команды, студент Осипенко Николай Николаевич, представленный объектом kolya класса Staff. У него зарегистрирован один Отпечаток пальца — объект bigFinger класса Fingerprint —, который наделён единственным Правилом доступа (объект rule класса Rule) — входить во второй корпус ТГУ (объект building2 класса Area) и выходить из него с 7:00 до 22:00 в любой день недели, за исключением воскресенья.

Полученная диаграмма объектов изображена на рисунке 5, а её характеристику можно найти в таблице 5.

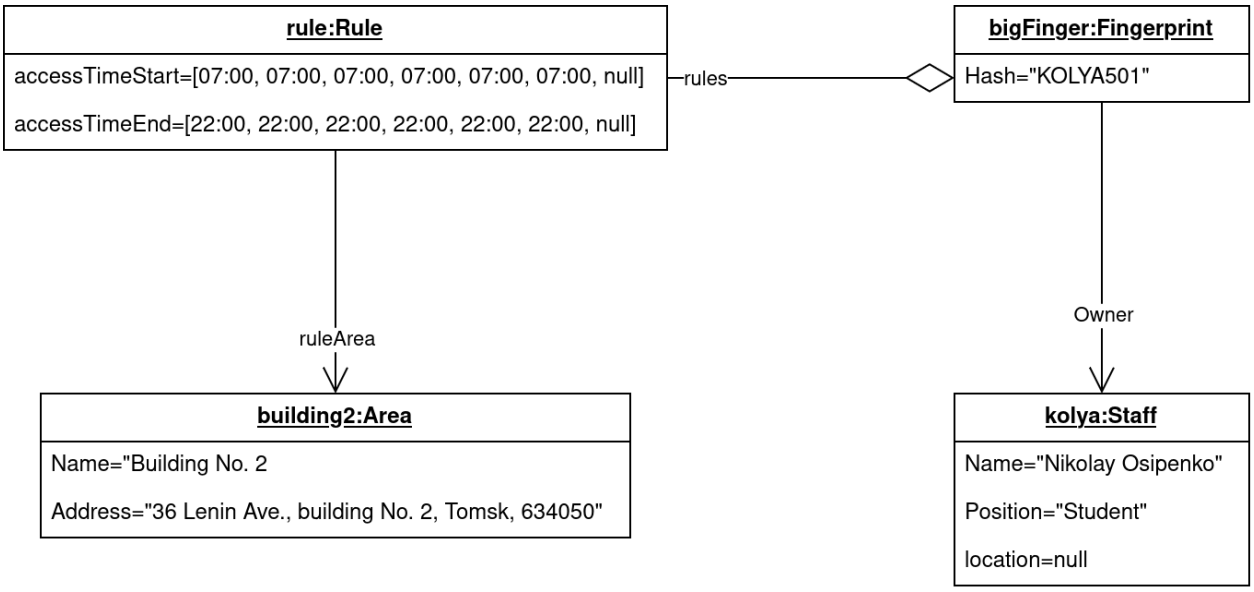


Рисунок 5 — диаграмма объектов для примера со студентом Осипенко Н. Н.

Таблица 5 — характеристика диаграммы объектов для выбранного примера

Вопрос	Как связаны конкретные объекты Сотрудника, его Отпечатка пальца, его Правила доступа и его Зоны доступа в заданном примером состоянии системы?
Читатель	Тестировщик системы контроля и управления доступом.
Что сделает читатель?	Проверит корректность связей между объектами и (корректность) исходных данных для тестового сценария, а также использует представленное состояние объектов как основу для тестирования предоставления доступа Сотрудникам во второй корпус ТГУ.
Что показано на схеме?	1. Конкретные экземпляры классов Fingerprint, Staff, Rule и Area с заданными значениями атрибутов, взятыми из примера со студентом Осипенко Н. Н.; 2. Последовательность, в которой должны создаваться объекты: bigFinger не может быть создан раньше, чем kolya, т.к. требует его в своём конструкторе, а rule — раньше, чем building2 (по тем же причинам) (ещё лучше это отслеживается в программном коде на языке программирования C#); 3. Возможность наличия null-значений в коллекциях accessTimeStart и accessTimeEnd у класса Rule (это, однако, не меняет их постоянного

	размера, равного 7), что соответствует полному запрету на доступ в указанную Зону доступа в указанный день недели.
Что не показано?	1. Процесс формирования хэша Отпечатка пальца; 2. Изменение состояния объектов во времени, включая перемещение Сотрудника (студента Осипенко Н. Н.), запрос им доступа в единственно возможную Зону доступа — второй корпус ТГУ — и регистрацию События прохода в Журнале прохода.
Допущения	Не выявлены.